

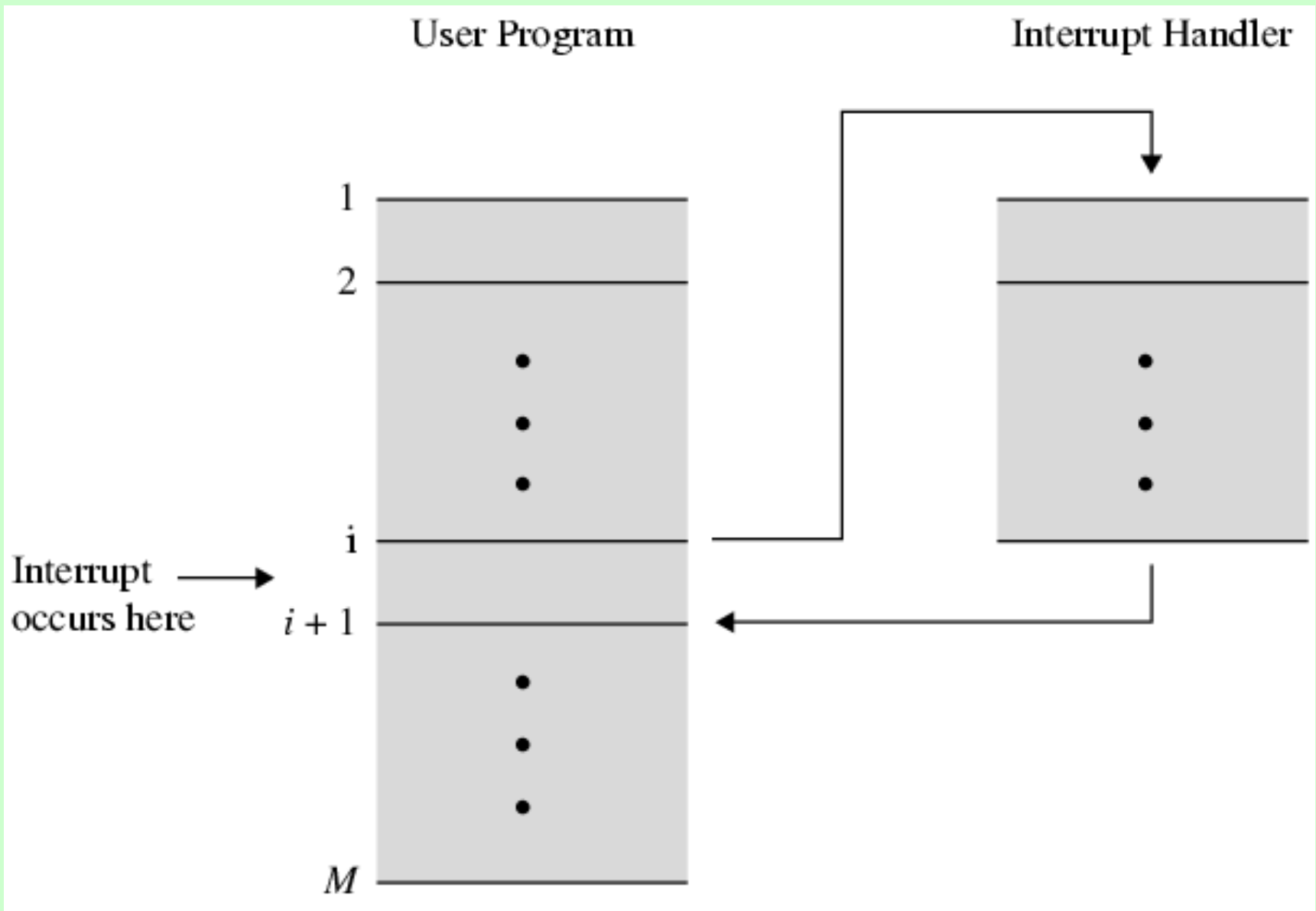
Interrupts

- Mechanism by which other modules (e.g. I/O) may interrupt normal sequence of processing
- Program
 - e.g. overflow, division by zero
- Timer
 - Generated by internal processor timer
 - Used in pre-emptive multi-tasking
- I/O
 - from I/O controller
- Hardware failure
 - e.g. memory parity error

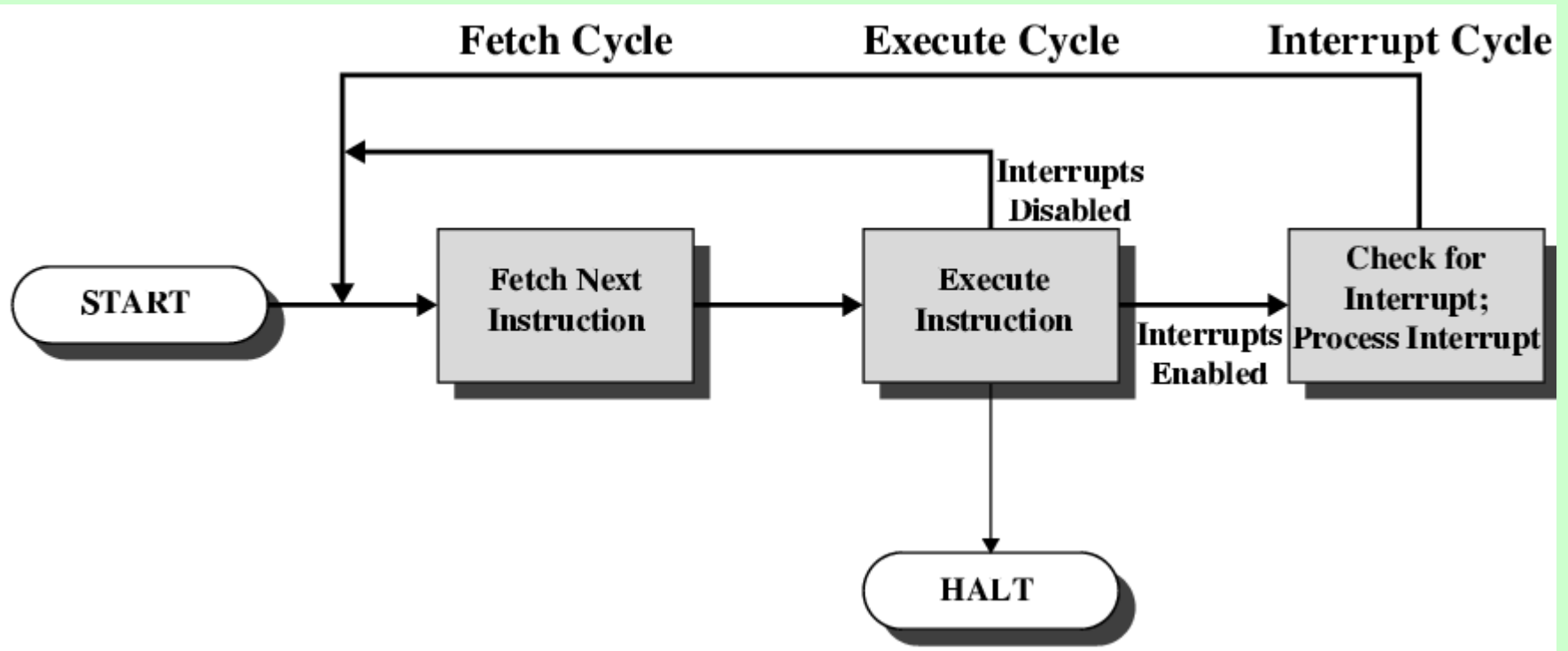
Interrupt Cycle

- Added to instruction cycle
- Processor checks for interrupt
 - Indicated by an interrupt signal
- If no interrupt, fetch next instruction
- If interrupt pending:
 - Suspend execution of current program
 - Save context
 - Set PC to start address of interrupt handler routine
 - Process interrupt
 - Restore context and continue interrupted program

Transfer of Control via Interrupts



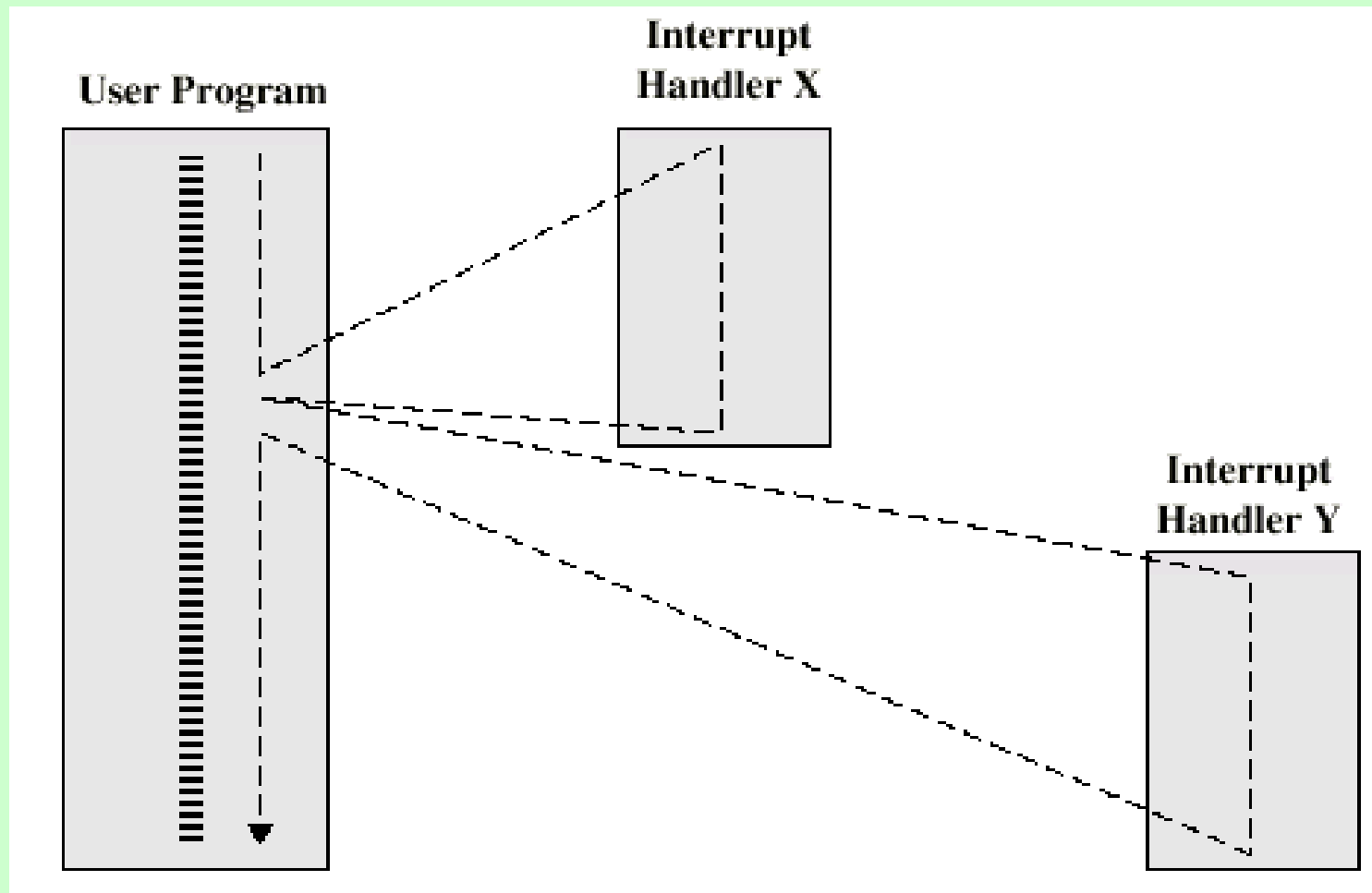
Instruction Cycle with Interrupts



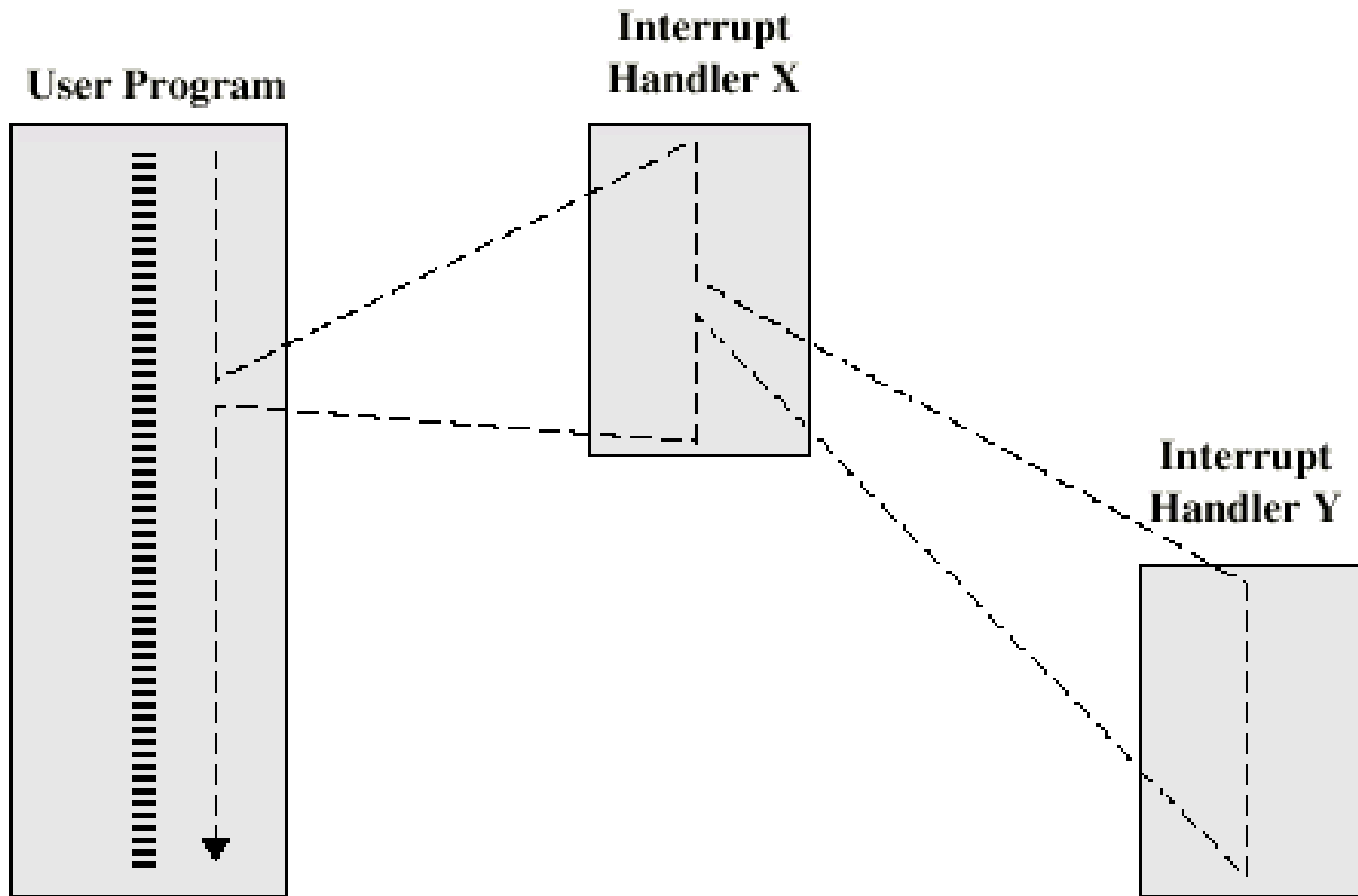
Multiple Interrupts

- Disable interrupts
 - Processor will ignore further interrupts whilst processing one interrupt
 - Interrupts remain pending and are checked after first interrupt has been processed
 - Interrupts handled in sequence as they occur
- Define priorities
 - Low priority interrupts can be interrupted by higher priority interrupts
 - When higher priority interrupt has been processed, processor returns to previous interrupt

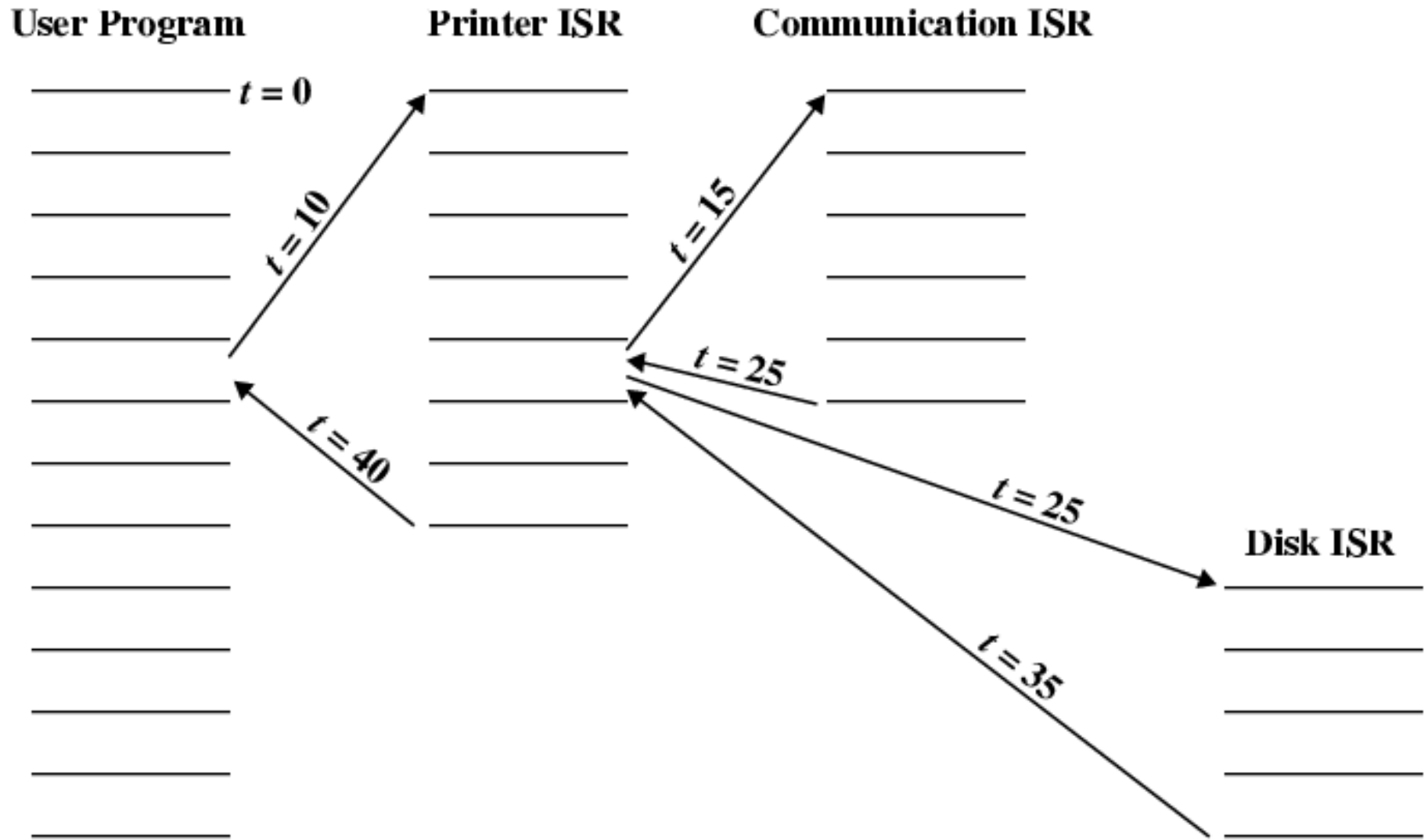
Multiple Interrupts - Sequential



Multiple Interrupts – Nested



Time Sequence of Multiple Interrupts





EC-220 Computer System Architecture

Dr. Ahsan Shahzad

Department of Computer and Software Engineering (DC&SE)

College of Electrical and Mechanical Engineering (CEME)

National University of Sciences and Technology (NUST)

Week # 16_OLE

Exceptions or Interrupts

- **Control is the most challenging aspect of processor design.**
 - One of the hardest parts of control is implementing exceptions and interrupts.
 - Exception
 - Also called interrupt. An unscheduled event that disrupts program execution; used to detect overflow.
 - Interrupt
 - An exception that comes from outside of the processor. (Some architectures use the term interrupt for all exceptions.)

Type of event	From where?	MIPS terminology
I/O device request	External	Interrupt
Invoke the operating system from user program	Internal	Exception
Arithmetic overflow	Internal	Exception
Using an undefined instruction	Internal	Exception
Hardware malfunctions	Either	Exception or interrupt

Exceptions Handling in MIPS Architecture

- **Control implementation for detecting two types of exceptions**
 - Undefined instruction
 - Arithmetic overflow
- How Exceptions are handled in the MIPS Architecture?
 - Example: Arithmetic overflow in `add $1, $2, $1`
 - EPC: save the address of the offending instruction in the exception program counter
 - Transfer control to the operating system (OS) at some specified address
 - OS can take the appropriate actions; predefined actions, or stopping the execution, reporting an error.
 - Lastly, OS can resume execution of user program using EPC or can terminate it.

Exceptions Handling in MIPS Architecture

- For OS to handle the exception, it must know the reason for the exception.

Two main methods:

- Status Register

- Include a status register called the Cause Register (used in MIPS Architecture)
- *Cause Register*: A register used to record the cause of the exception. In the MIPS architecture, this register is 32 bits, although some bits are currently unused.
- Thus, control transfer to OS will be done using a fix address irrespective of exception type.

- Vectored Interrupts

- An interrupt for which the address to which control is transferred is determined by the cause of the exception.
- Addresses are separated by 32 bytes or eight instructions; limited processing code sequence

Exception type	Exception vector address (in hex)
Undefined instruction	8000 0000 _{hex}
Arithmetic overflow	8000 0180 _{hex}

Exceptions in a Pipelined Implementation

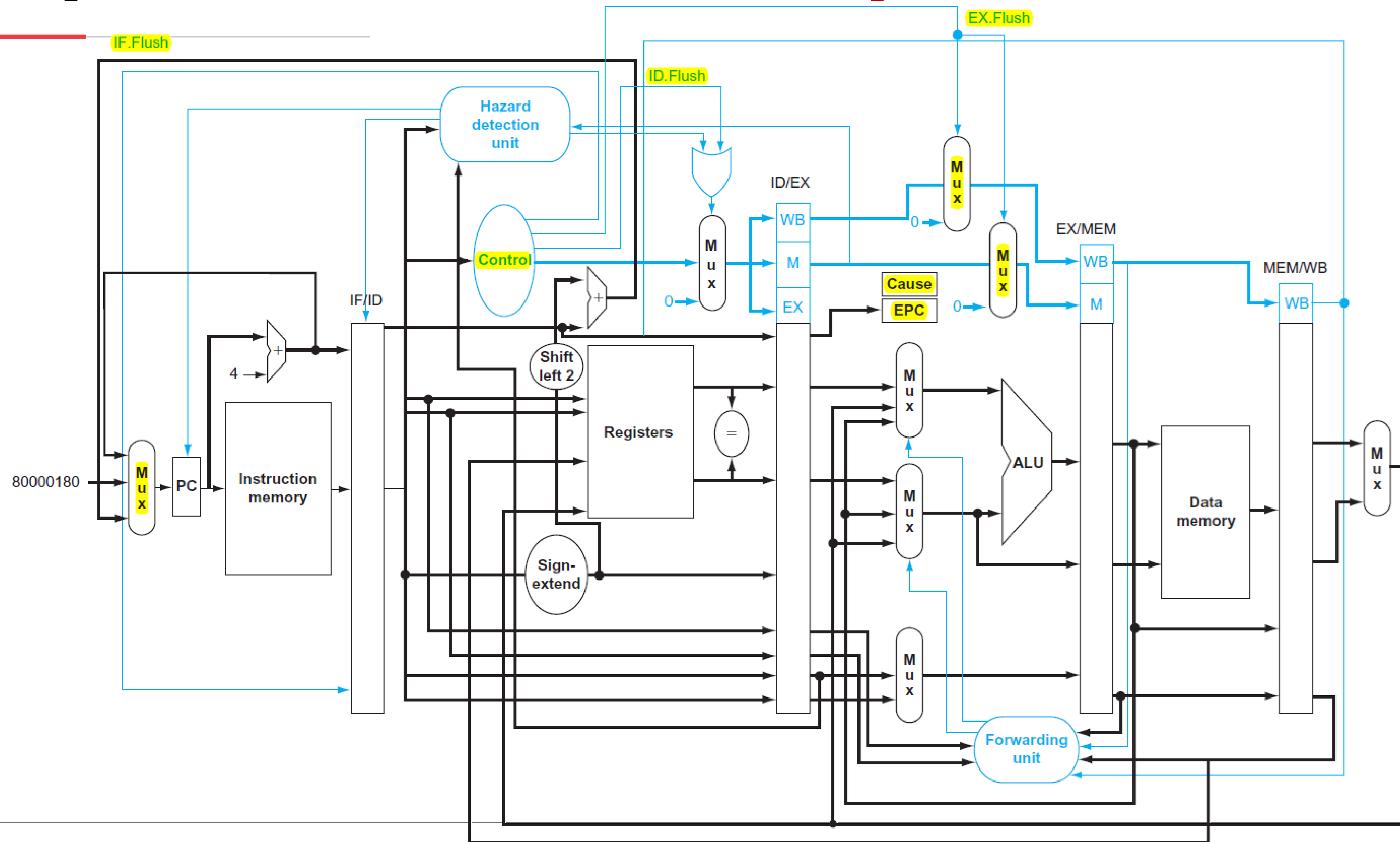
- **Another form of Control Hazard**

- For Branch mispredict, flush the instruction in IF stage by turning it into *nop*.

- **Exceptions Handling**

- Flush the offending instruction and the following ones from the pipeline
- At IF stage, similar to branch by converting it a nop
- To flush at ID stage: zeros the control signals i.e. inserting a bubble, new control signal ID.Flush ORed with Stall signal from Hazard detection unit.
- To flush at EX stage: new control signal EX.Flush will make the remaining control signals zero.
- Start fetching instructions from 8000 0810 hex (single entry point in MIPS exceptions). Require additional input to PC reg. MUX
- Final step is to save the address of offending instruction in the EPC. In reality we are saving address + 4, which should be corrected by exception handling routine/software by subtracting 4.

Datapath with Controls to **handle Exceptions**



Example: Exception in a Pipelined Computer

Example:

Given this instruction sequence,

40 _{hex}	sub	\$11, \$2, \$4
44 _{hex}	and	\$12, \$2, \$5
48 _{hex}	or	\$13, \$2, \$6
4C _{hex}	add	\$1, \$2, \$1
50 _{hex}	slt	\$15, \$6, \$7
54 _{hex}	lw	\$16, 50(\$7)
...		

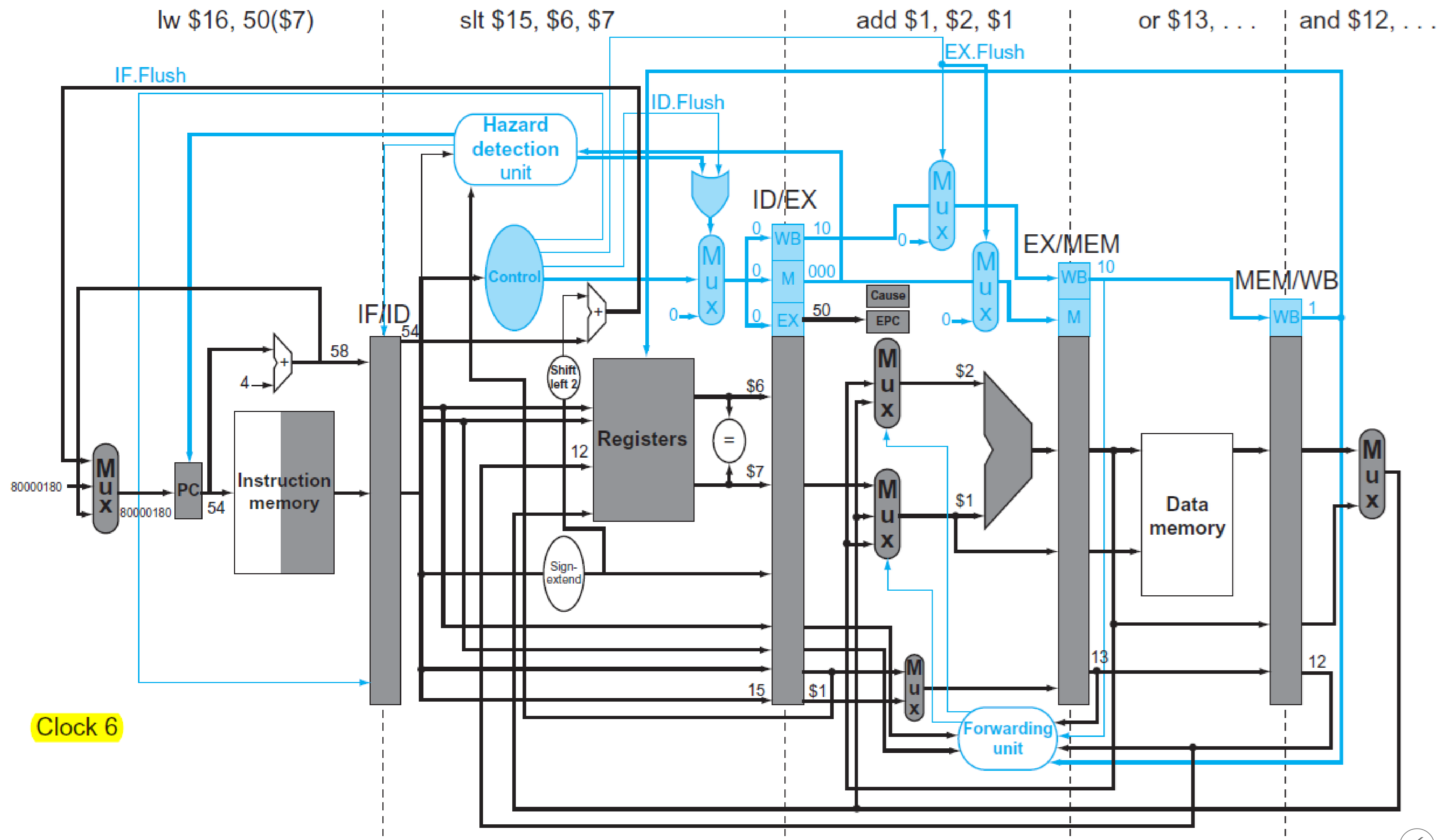
assume the instructions to be invoked on an exception begin like this:

80000180 _{hex}	sw	\$26, 1000(\$0)
80000184 _{hex}	sw	\$27, 1004(\$0)
...		

Show what happens in the pipeline if an overflow exception occurs in the add instruction.

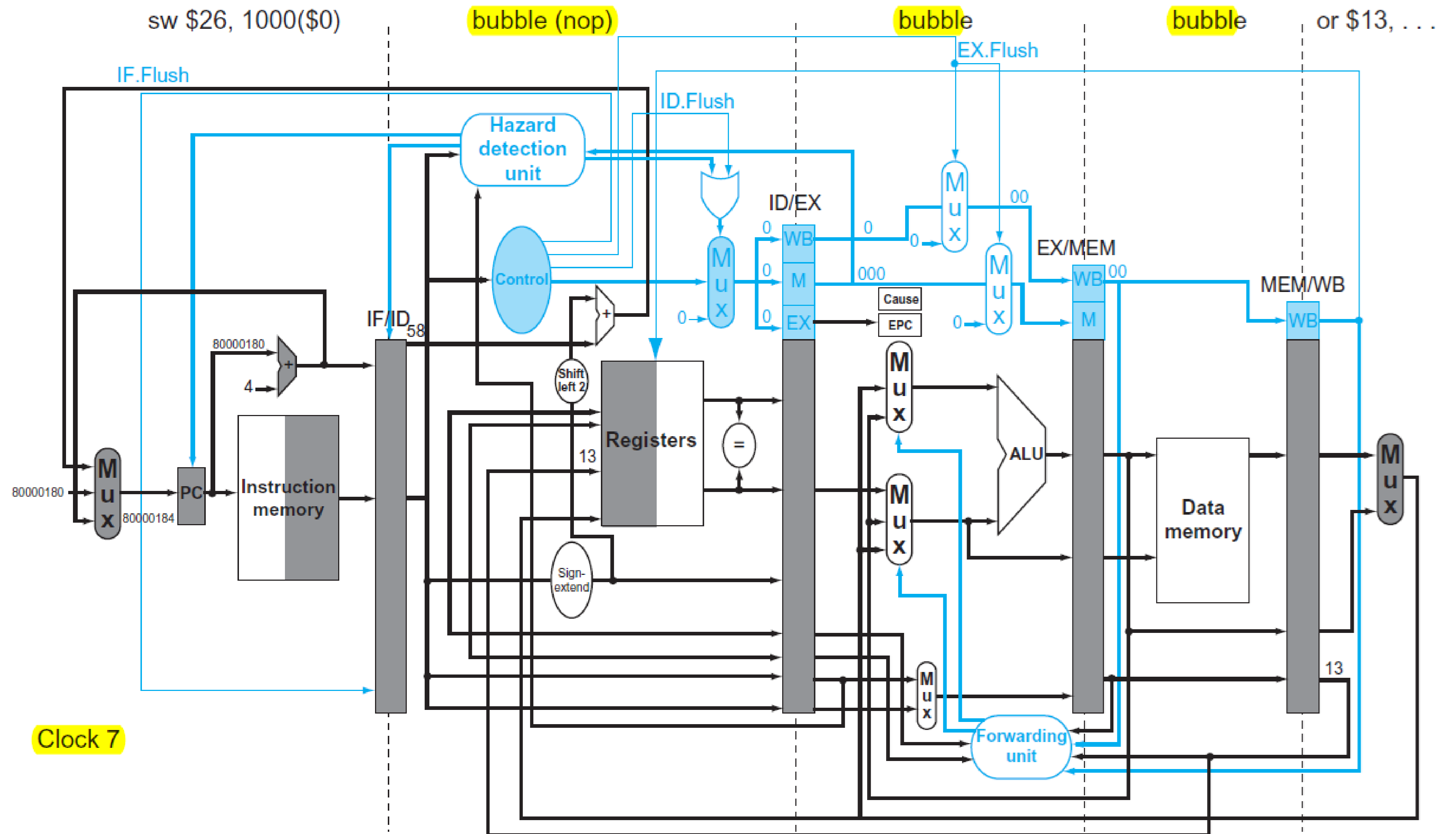
Example: Exception in a Pipelined Computer

Example:



Example: Exception in a Pipelined Computer

Example:



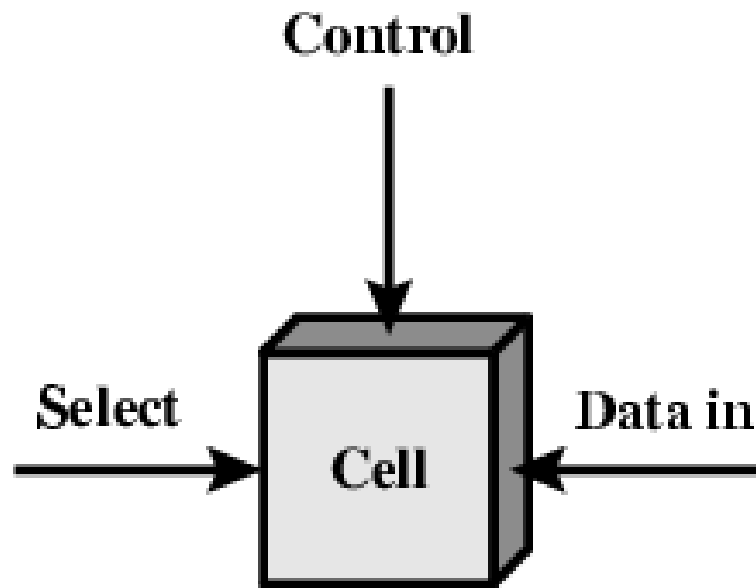
Semiconductor Memory Types

Memory Type	Category	Erasure	Write Mechanism	Volatility
Random-access memory (RAM)	Read-write memory	Electrically, byte-level	Electrically	Volatile
Read-only memory (ROM)	Read-only memory	Not possible	Masks	Nonvolatile
Programmable ROM (PROM)			Electrically	
Erasable PROM (EPROM)	Read-mostly memory	UV light, chip-level		
Electrically Erasable PROM (EEPROM)		Electrically, byte-level		
Flash memory		Electrically, block-level		

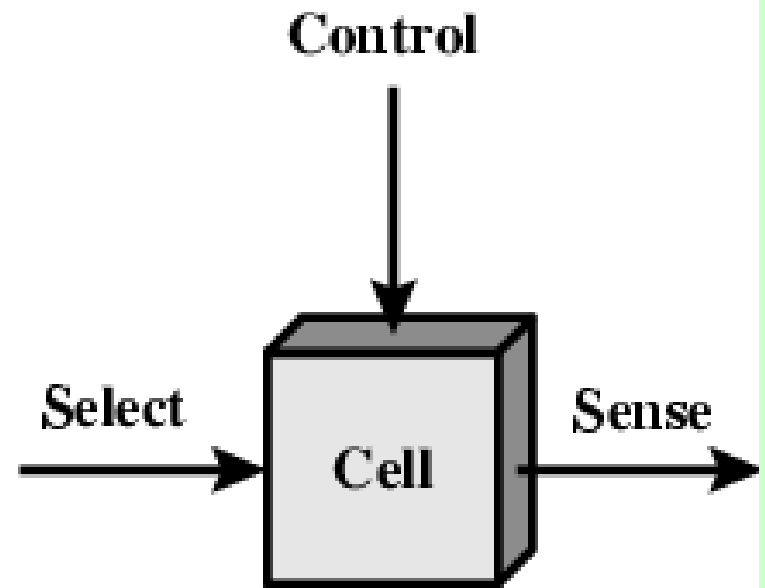
Semiconductor Memory

- RAM
 - Misnamed as all semiconductor memory is random access
 - Read/Write
 - Volatile
 - Temporary storage
 - Static or dynamic

Memory Cell Operation



(a) Write

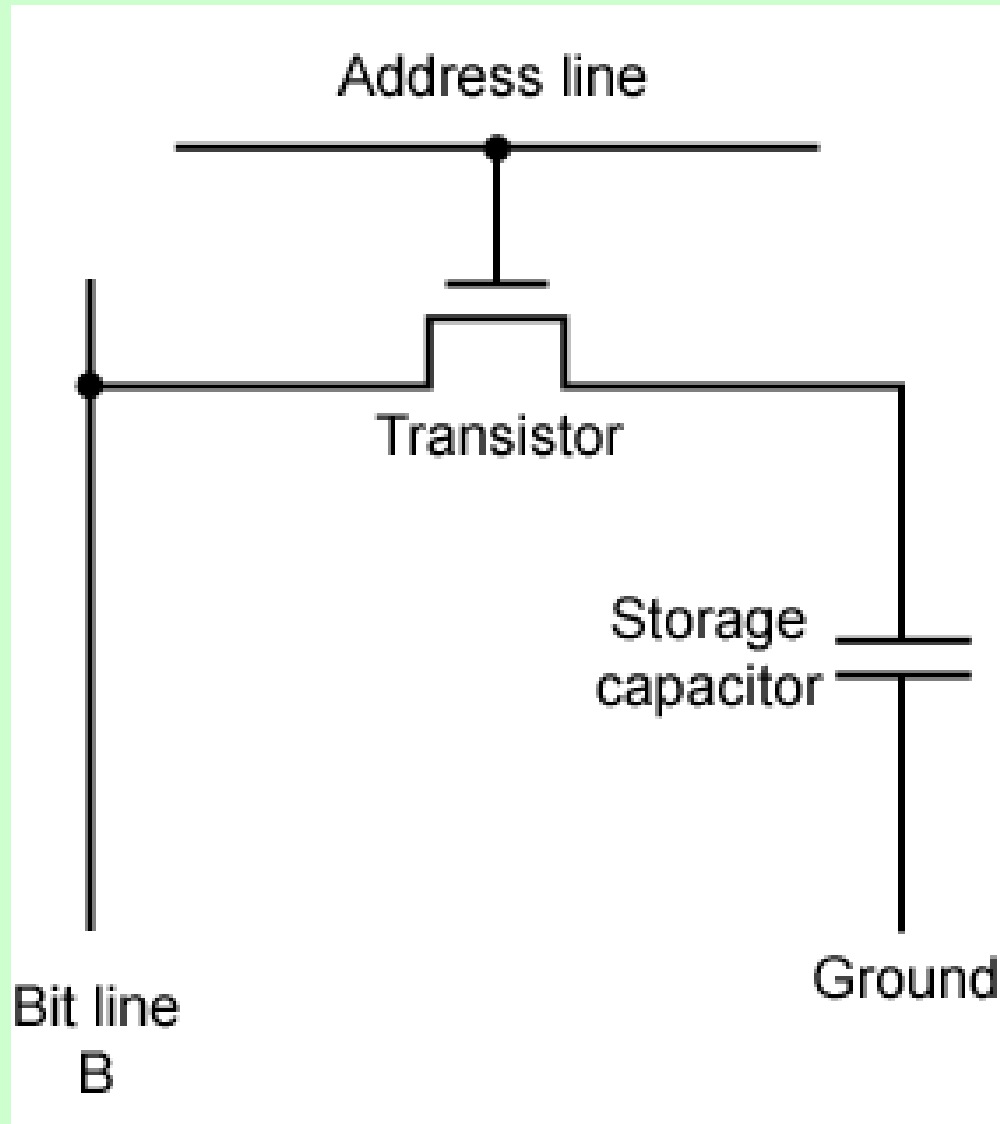


(b) Read

Dynamic RAM

- Bits stored as charge in capacitors
- Charges leak
- Need refreshing even when powered
- Simpler construction
- Smaller per bit
- Less expensive
- Need refresh circuits
- Slower
- Main memory
- Essentially analogue
 - Level of charge determines value

Dynamic RAM Structure



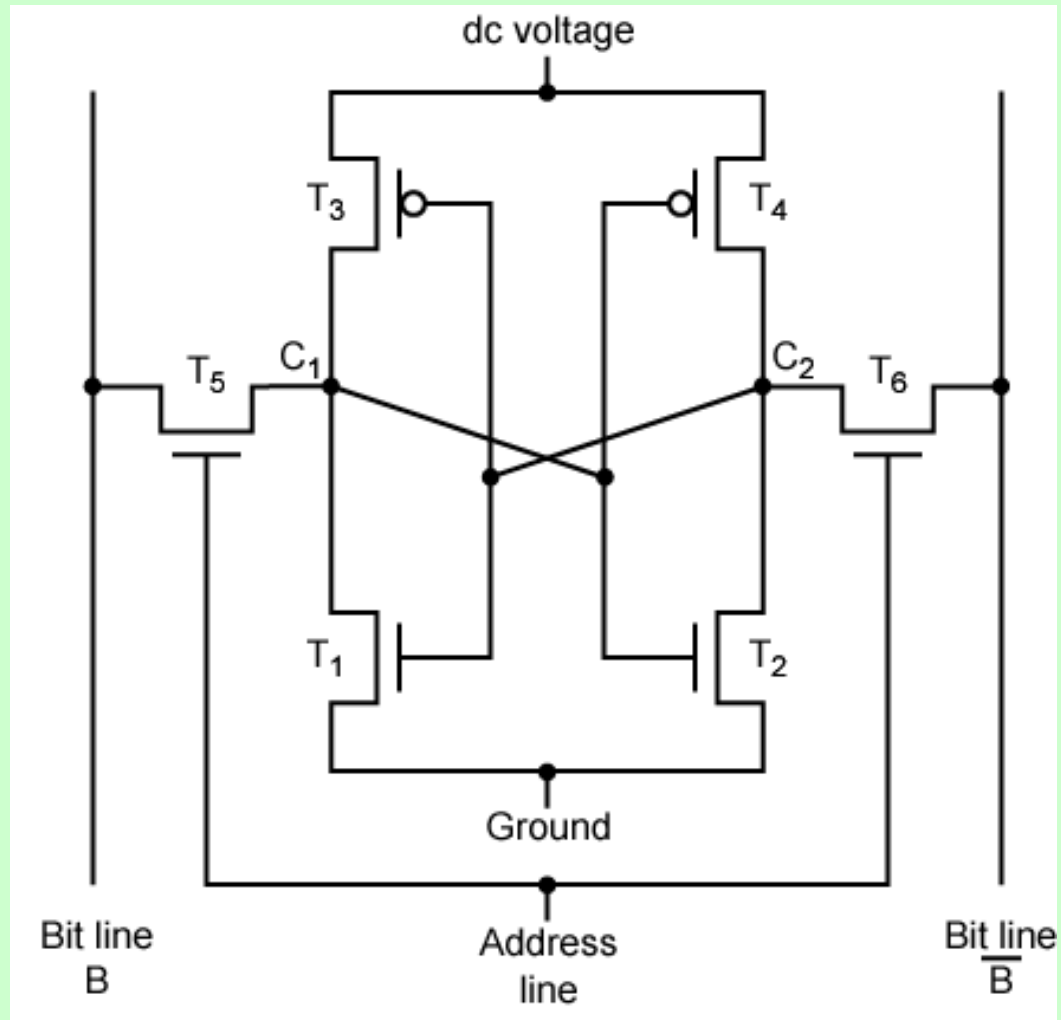
DRAM Operation

- Address line active when bit read or written
 - Transistor switch closed (current flows)
- Write
 - Voltage to bit line
 - High for 1 low for 0
 - Then signal address line
 - Transfers charge to capacitor
- Read
 - Address line selected
 - transistor turns on
 - Charge from capacitor fed via bit line to sense amplifier
 - Compares with reference value to determine 0 or 1
 - Capacitor charge must be restored

Static RAM

- Bits stored as on/off switches
- No charges to leak
- No refreshing needed when powered
- More complex construction
- Larger per bit
- More expensive
- Does not need refresh circuits
- Faster
- Cache
- Digital
 - Uses flip-flops

Stating RAM Structure



Static RAM Operation

- Transistor arrangement gives stable logic state
- State 1
 - C_1 high, C_2 low
 - T_1 T_4 off, T_2 T_3 on
- State 0
 - C_2 high, C_1 low
 - T_2 T_3 off, T_1 T_4 on
- Address line transistors T_5 T_6 is switch
- Write – apply value to B & compliment to $\overline{B}$
- Read – value is on line B

SRAM v DRAM

- Both volatile
 - Power needed to preserve data
- Dynamic cell
 - Simpler to build, smaller
 - More dense
 - Less expensive
 - Needs refresh
 - Larger memory units
- Static
 - Faster
 - Cache

Read Only Memory (ROM)

- Permanent storage
 - Nonvolatile
- Microprogramming (see later)
- Library subroutines
- Systems programs (BIOS)
- Function tables

Types of ROM

- Written during manufacture
 - Very expensive for small runs
- Programmable (once)
 - PROM
 - Needs special equipment to program
- Read “mostly”
 - Erasable Programmable (EPROM)
 - Erased by UV
 - Electrically Erasable (EEPROM)
 - Takes much longer to write than read
 - Flash memory
 - Erase whole memory electrically

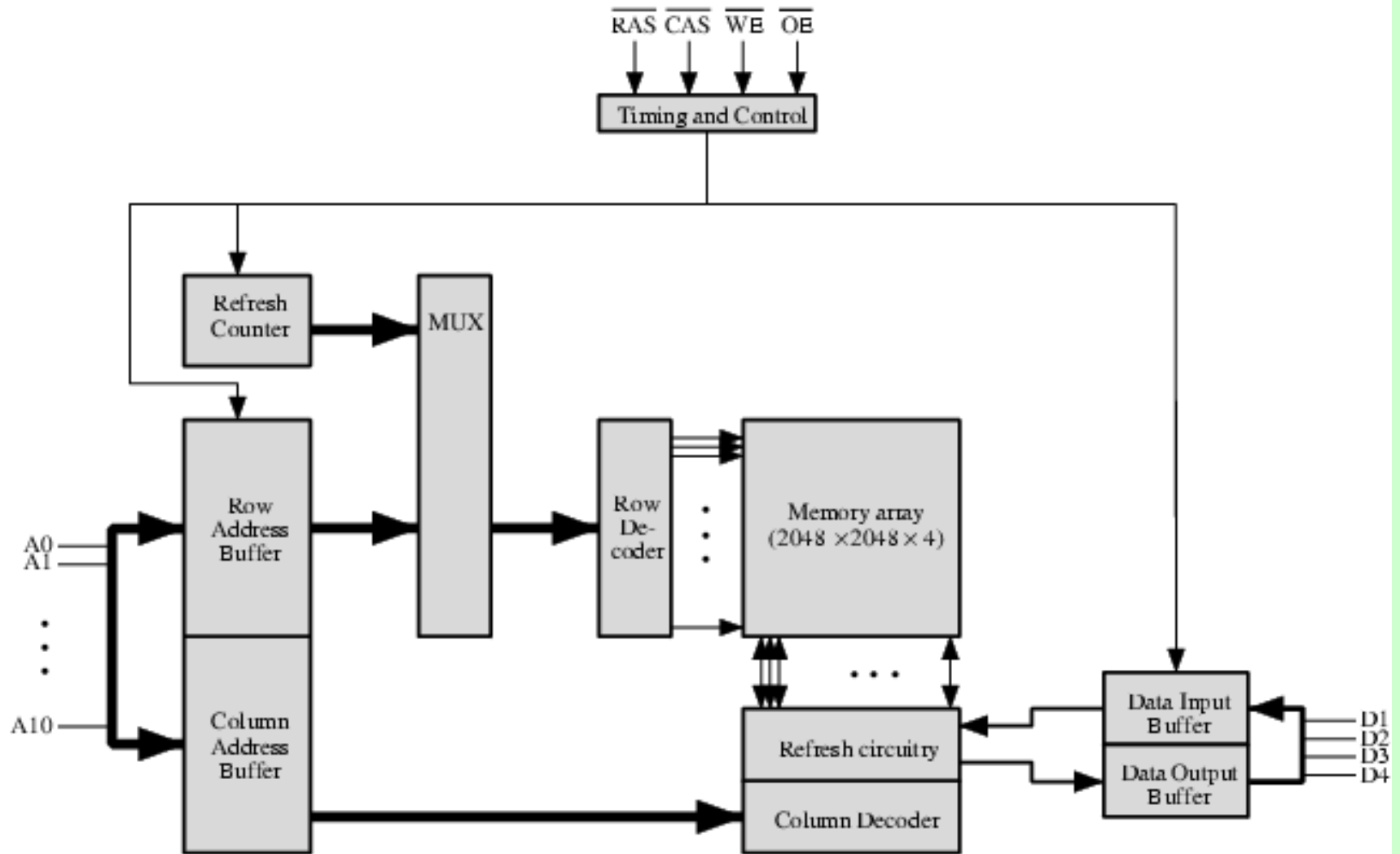
Organisation in detail

- A 16Mbit chip can be organised as 1M of 16 bit words
- A bit per chip system has 16 lots of 1Mbit chip with bit 1 of each word in chip 1 and so on
- A 16Mbit chip can be organised as a 2048 x 2048 x 4bit array
 - Reduces number of address pins
 - Multiplex row address and column address
 - 11 pins to address ($2^{11}=2048$)
 - Adding one more pin doubles range of values so x4 capacity

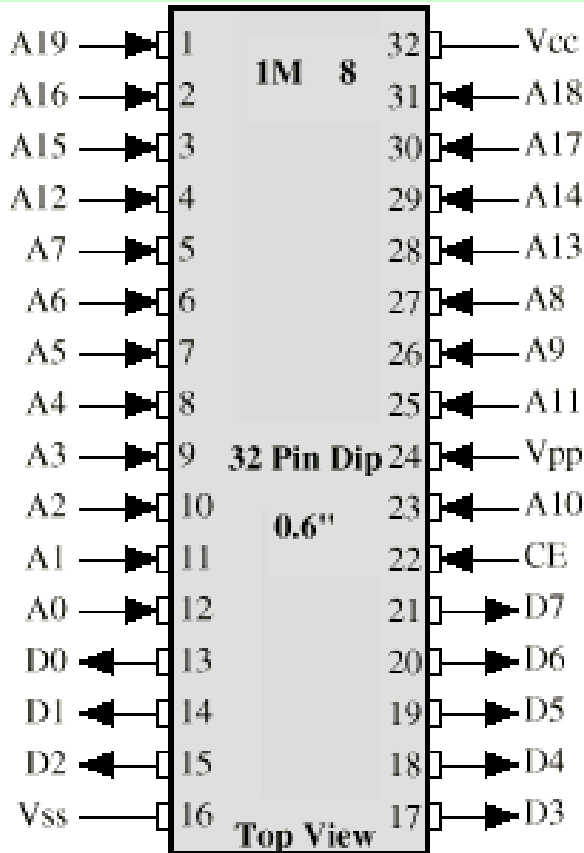
Refreshing

- Refresh circuit included on chip
- Disable chip
- Count through rows
- Read & Write back
- Takes time
- Slows down apparent performance

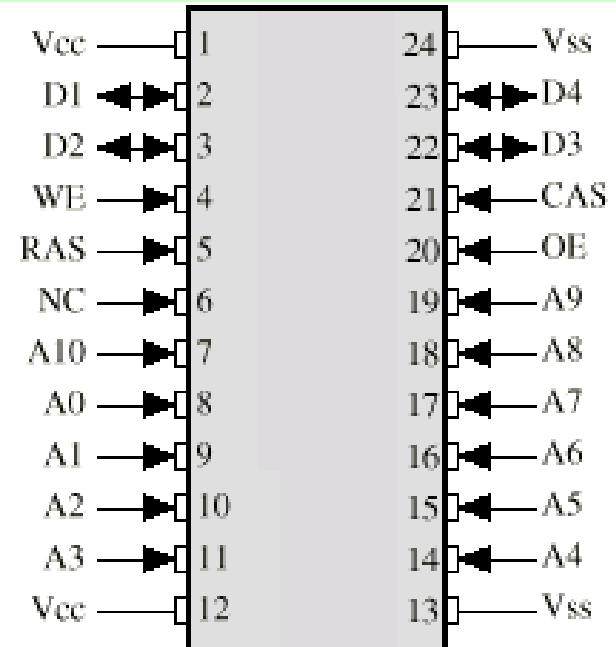
Typical 16 Mb DRAM (4M x 4)



Packaging

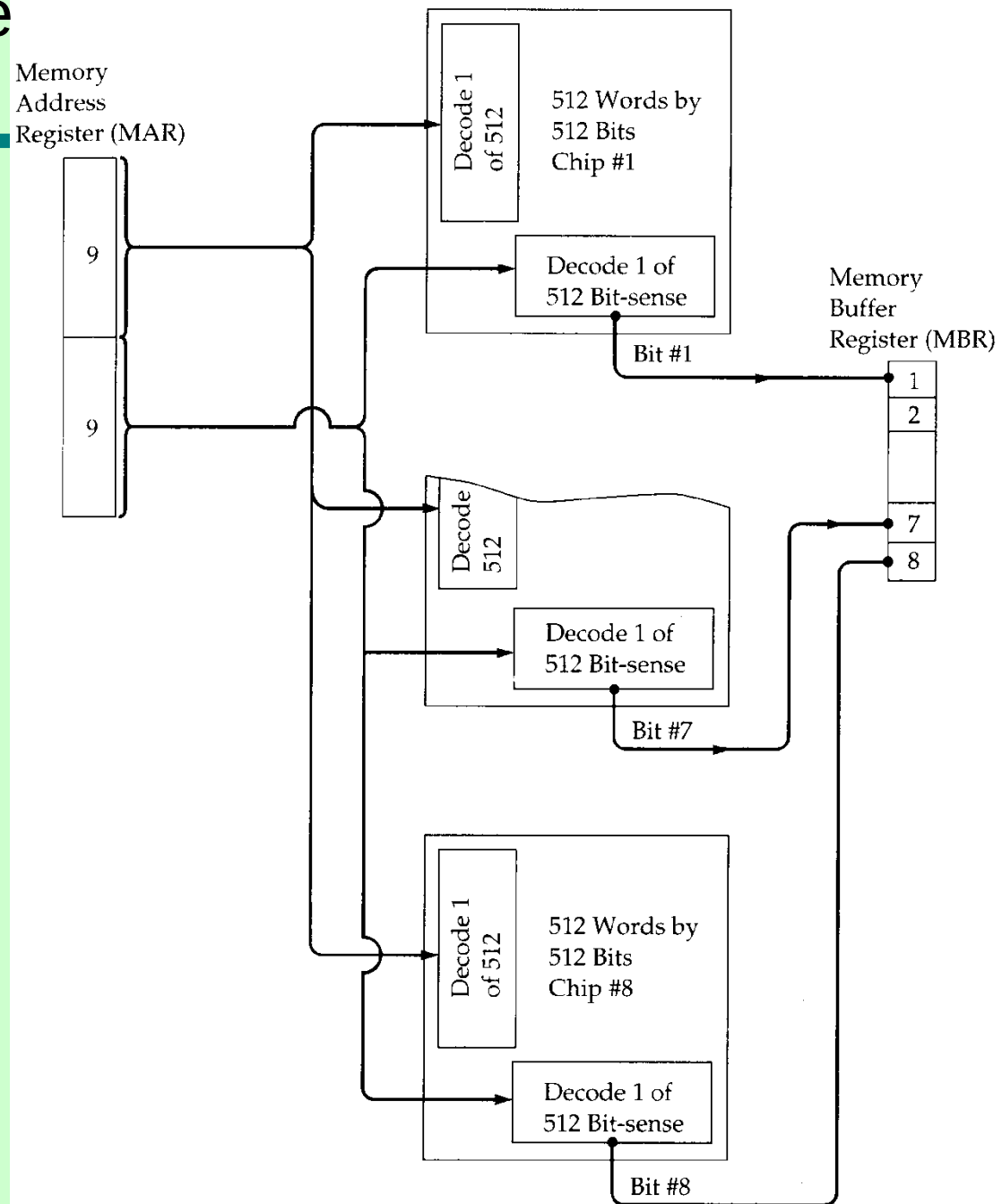


(a) 8 Mbit EPROM



(b) 16 Mbit DRAM

256kByte Module Organisation



Interleaved Memory

- Collection of DRAM chips
- Grouped into memory bank
- Banks independently service read or write requests
- K banks can service k requests simultaneously

Error Correction

- Hard Failure
 - Permanent defect
- Soft Error
 - Random, non-destructive
 - No permanent damage to memory
- Detected using Hamming error correcting code

Simplified DRAM Read Timing

